

VALUE ADDED COURSE ON AI/ ML - TASK 3

NAME: HARSHIT KUMAR

USN: 24BBCAG015

COURSE: BCA (GENERAL)

SEMESTER: 5TH / YEAR: III

SUBJECT: ARTIFICIAL INTELLIGENCE

EXPERIMENT 3: SMS/Email Spam & Phishing Detector using Natural Language Processing

AIM:

To preprocess SMS text using **TF-IDF**, train a **Multinomial Naive Bayes** model, and detect spam/phishing messages using **Accuracy, Precision, Recall, and Confusion Matrix**.

SYSTEM REQUIREMENTS:

Hardware Requirements:

- Processor: Intel Core i3 or equivalent (minimum)
- RAM: 4 GB or higher
- Internet Connection: Required (for loading raw dataset directly in Google Colab)

Software Requirements:

- Operating System: Windows 10/11, macOS, or Linux
- Platform: Google Colab / Jupyter Notebook
- Environment: Python 3.x
- Required Libraries: pandas, numpy, scikit-learn

PROCEDURE:

1. **Dataset Acquisition:** Load the UCI/Kaggle SMS Spam dataset from a public URL.
2. **Data Cleaning:** Select and rename the required label and message columns.
3. **Target Encoding:** Convert **ham** → **0** and **spam** → **1**.
4. **Text Vectorization:** Convert messages into a **TF-IDF matrix** using stop-word removal.
5. **Train-Test Split:** Split the data into **80% training and 20% testing** using stratified sampling.
6. **Model Training:** Train a **Multinomial Naive Bayes** classifier on the training data.
7. **Evaluation & Inference:** Evaluate the model using **Accuracy, Precision, Recall, and Confusion Matrix**, and test it with custom messages.

PROGRAM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 1: Load Dataset
DATA_URL = "https://raw.githubusercontent.com/mohitgupta-1O1/Kaggle-SMS-Spam-Collection-Dataset-/master/spam.csv"
df = pd.read_csv(DATA_URL, encoding='latin-1')

# Step 2: Clean Dataset
df = df[['v1', 'v2']]
df.columns = ['Label', 'Message']

print("--- Step 1: Dataset Overview ---")
print(f"Total Samples: {df.shape[0]}\n")
print(df.head())
print("\n" + "=" * 60 + "\n")

# Step 3: Target Encoding
df['Target'] = df['Label'].map({'ham': 0, 'spam': 1})

X_raw = df['Message']
y = df['Target']

# Step 4: TF-IDF Vectorization
tfidf = TfidfVectorizer(stop_words='english', lowercase=True)
```

```
X_tfidf = tfidf.fit_transform(X_raw)

print(f"TF-IDF Feature Matrix Shape: {X_tfidf.shape}")
print("=" * 60 + "\n")

# Step 5: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, y, test_size=0.2, random_state=42, stratify=y
)

# Step 6: Model Training
model = MultinomialNB()
model.fit(X_train, y_train)

# Step 7: Prediction and Evaluation
y_pred = model.predict(X_test)

acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("--- Step 2: Model Evaluation ---")
print(f"Model Accuracy: {acc * 100:.2f}%")
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred,
    target_names=['Ham (Legit)', 'Spam/Phishing']
))

# Step 8: Custom Message Prediction
```

```
def predict_spam(message):
    vectorized_msg = tfidf.transform([message])
    prediction = model.predict(vectorized_msg)[0]
    probability = model.predict_proba(vectorized_msg)[0][1]

    result = "SPAM / PHISHING DETECTED" if prediction == 1 else "LEGITIMATE (HAM)"

    print(f"\nMessage: {message}")
    print(f"Result: {result}")
    print(f"Spam Probability: {probability * 100:.1f}%")

print("=" * 60)
print("--- Step 3: Real-Time Test Predictions ---")

predict_spam(
    "URGENT! You have won a $1,000 Walmart gift card. Click link to claim now!"
)

predict_spam(
    "Hey boss, let me know when you are free for the project meeting today."
)

print("=" * 60)
```

OUTPUT:

```
--- Step 1: Dataset Overview ---
Total Samples: 5572

  Label                                            Message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                      Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

============================================================

TF-IDF Feature Matrix Shape: (5572, 8404)
============================================================

--- Step 2: Model Evaluation ---
Model Accuracy: 97.22%

Confusion Matrix:
[[966   0]
 [ 31 118]]

Classification Report:
              precision    recall  f1-score   support

  Ham (Legit)       0.97      1.00      0.98       966
Spam/Phishing       1.00      0.79      0.88       149

     accuracy                           0.97      1115
    macro avg       0.98      0.90      0.93      1115
 weighted avg       0.97      0.97      0.97      1115

============================================================
--- Step 3: Real-Time Test Predictions ---

Message: URGENT! You have won a $1,000 Walmart gift card. Click link to claim now!
Result: SPAM / PHISHING DETECTED
Spam Probability: 75.3%

Message: Hey boss, let me know when you are free for the project meeting today.
Result: LEGITIMATE (HAM)
Spam Probability: 0.3%
============================================================
```

RESULT:

The SMS Spam and Phishing Detection model was successfully trained using **TF-IDF and Multinomial Naive Bayes**. The model classified messages as **legitimate or spam/phishing** and was evaluated using **Accuracy, Precision, Recall, and Confusion Matrix**.